NeetCode 150

Python revision notes · summary, key idea, complexity and solution per problem

Author: Claude Opus 5.5

Problems	Categories	Easy	Medium	Hard
150	18	28	100	22

How to revise with this

- Read the **problem** line and try to recall the approach before reading the **key idea**. If it comes to you, skim the code; if not, read it slowly and re-derive why it works.
- The **pattern cheat sheet** is the fastest pass the night before a test: one block per category with the trigger, the template and the classic traps.
- The contents and PDF bookmarks link to every category and problem.
- Problem summaries are paraphrased, not copied from LeetCode; check the original statement for exact constraints.
- Add your own reminders inside any problem with `\note{...}`.

Shared definitions assumed by the solutions

Solutions use LeetCode's signatures. These imports and node classes are provided by LeetCode and omitted from each problem.

```python
from typing import List, Optional
from collections import defaultdict, Counter, deque
import heapq, bisect, functools

class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

Problem list follows the classic NeetCode 150 grouping (18 categories); the live site may differ slightly. Every solution was run against test cases (LeetCode examples plus edge cases) before compiling; all passed.

Pattern cheat sheet

One block per category: when to reach for it, the core move, a template, and the traps.

Arrays & Hashing

Spot it: Need O(1) lookup, counting, grouping, or 'have I seen this?'

Core move: Hash set / hash map / Counter. Group by a canonical key (sorted string, count tuple). Prefix and suffix products/sums.

Traps: Lists are unhashable: use tuple(count) as a key. Sequence-start trick: only count from n where n-1 is absent.

```
seen = {}
for i, x in enumerate(nums):
    if target - x in seen: ...
    seen[x] = i
```

Two Pointers

Spot it: Sorted input, pair/triplet sums, palindromes, or shrinking from both ends.

Core move: l at start, r at end; move the pointer that can improve the answer.

Traps: Skip duplicates after a match (3Sum). Move the shorter wall (Container, Trap).

```
l, r = 0, len(a) - 1
while l < r:
    if cond: l += 1
    else: r -= 1
```

Sliding Window

Spot it: Longest/shortest contiguous substring or subarray satisfying a condition.

Core move: Grow r every step; shrink l while the window is invalid; record the answer.

Traps: Fixed-size window: slide by adding r and removing r-k. Window max: monotonic deque.

```
l = 0
for r in range(len(s)):
    add(s[r])
    while invalid(): remove(s[l]); l += 1
    best = max(best, r - l + 1)
```

Stack

Spot it: Matching pairs, 'next greater/warmer', expression evaluation, nested structure.

Core move: Push openers / indices; pop when the current element resolves the top. Monotonic stack for next-greater.

Traps: RPN division in Python: int(a / b), not a // b. Histogram: popped bar's start carries over.

```
st = []
for i, x in enumerate(a):
    while st and a[st[-1]] < x:
        j = st.pop(); res[j] = i - j
    st.append(i)
```

Binary Search

Spot it: Sorted data, rotated arrays, or 'minimum X such that condition holds'.

Core move: Halve the search space; on answer-space problems search over the answer value.

Traps: Rotated array: one half is always sorted; check that half first. Median: partition the shorter array.

```
l, r = lo, hi
while l <= r:
    m = (l + r) // 2
    if ok(m): res = m; r = m - 1
    else: l = m + 1
```

Linked List

Spot it: In-place pointer rewiring, middle of list, cycles, k-th from end.

Core move: Dummy head; slow/fast pointers; reverse with prev/cur/nxt.

Traps: Find the Duplicate: array as a linked list (Floyd). LRU: hash map + doubly linked list with sentinels.

```
prev, cur = None, head
while cur:
    nxt = cur.next; cur.next = prev
    prev, cur = cur, nxt
```

Trees

Spot it: Anything recursive over a hierarchy: depth, paths, validation, traversal order.

Core move: DFS returning a value up (height, best arm), or BFS by level with a queue.

Traps: BST: in-order is sorted; validate with (lo, hi) bounds, not just parent. Max path sum: clamp arms at 0.

```
def dfs(node):
    if not node: return 0
    l, r = dfs(node.left), dfs(node.right)
    # update global answer with l, r
    return 1 + max(l, r)
```

Tries

Spot it: Prefix queries, word dictionaries, searching many words at once.

Core move: Node = children dict + end flag. Walk character by character.

Traps: Word Search II: prune found words and empty branches or it times out.

```
node = root
for c in word:
    node = node.children.setdefault(c, TrieNode())
node.end = True
```

Heap / Priority Queue

Spot it: k largest/smallest, running median, merging sorted streams, scheduling.

Core move: heapq is a min-heap. For max-heap push negatives. Keep a heap of size k for top-k.

Traps: Median: two heaps (max-heap low half, min-heap high half). Tie-breakers in tuples avoid comparing nodes.

```
h = []
for x in nums:
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)
# h[0] is k-th largest
```

Backtracking

Spot it: 'Return all' combinations, subsets, permutations, placements.

Core move: Choose, recurse, un-choose. Sort first when duplicates must be skipped.

Traps: Append a copy (cur[:]). Skip equal neighbours at the same depth for duplicate inputs.

```
def dfs(i):
    if done: res.append(cur[:]); return
    cur.append(x); dfs(i + 1); cur.pop()
    dfs(i + 1)
```

Graphs

Spot it: Grids, connectivity, islands, dependencies, shortest steps in unweighted graphs.

Core move: DFS/BFS flood fill; multi-source BFS for 'minutes/distance'; Kahn's topological sort; Union-Find.

Traps: Mark visited when enqueuing, not when popping. Reverse the flow when it is easier (Pacific Atlantic, Surrounded Regions).

```
q = deque(sources)
while q:
    for _ in range(len(q)):
        r, c = q.popleft()
        for nr, nc in neighbours(r, c): ...
    steps += 1
```

Advanced Graphs

Spot it: Weighted shortest paths, minimum spanning tree, ordering from constraints, Euler paths.

Core move: Dijkstra (heap), Prim (heap), Bellman-Ford (k rounds), Kahn (topo), Hierholzer (Euler path).

Traps: K stops: Bellman-Ford with a copy each round. Alien Dictionary: a word before its own prefix is invalid.

```
h = [(0, src)]; dist = {}
while h:
    d, u = heapq.heappop(h)
    if u in dist: continue
    dist[u] = d
    for v, w in adj[u]: heapq.heappush(h, (d + w, v))
```

1-D Dynamic Programming

Spot it: Count ways / min cost / max value where the answer at i depends on a few earlier states.

Core move: Define dp[i], write the recurrence, then compress to two variables if possible.

Traps: Palindromes: expand around centres. Max product: track min and max. LIS: patience sorting + bisect.

```
a, b = base0, base1
for x in arr:
    a, b = b, max(b, a + x)   # House Robber
```

2-D Dynamic Programming

Spot it: Two strings or a grid; choices at each (i, j); knapsack-style counting.

Core move: dp[i][j] over suffixes (or prefixes); fill bottom-up or memoise top-down.

Traps: Coin Change II: coins in the outer loop (combinations, not permutations). Burst Balloons: pick the LAST balloon.

```
for i in range(m - 1, -1, -1):
    for j in range(n - 1, -1, -1):
        dp[i][j] = f(dp[i+1][j],
                     dp[i][j+1],
                     dp[i+1][j+1])
```

Greedy

Spot it: A locally best move is provably safe: max subarray, reachability, partitioning.

Core move: Keep one running quantity (sum, goal, reach, open-count range) and update it each step.

Traps: Gas Station: total check first, then reset start. Parenthesis with *: track a [lo, hi] range.

```
cur = best = nums[0]
for x in nums[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
```

Intervals

Spot it: Overlaps, merging, inserting, rooms needed, queries over ranges.

Core move: Sort by start (or end); compare each interval's start with the previous end.

Traps: Non-overlapping: on conflict keep the smaller end. Meeting Rooms II: separate sorted starts and ends.

```
intervals.sort()
res = [intervals[0]]
for s, e in intervals[1:]:
    if s <= res[-1][1]:
        res[-1][1] = max(res[-1][1], e)
    else:
        res.append([s, e])
```

Math & Geometry

Spot it: Matrix manipulation, digit arithmetic, cycles in number sequences.

Core move: Transpose + reverse to rotate; boundary pointers for spirals; fast power by squaring.

Traps: Set Matrix Zeroes in O(1) space: first row/col as markers plus one flag. Negative exponents: invert x.

```
top, bottom, left, right = 0, R - 1, 0, C - 1
while top <= bottom and left <= right: ...
```

Bit Manipulation

Spot it: XOR cancellation, counting bits, arithmetic without + or -.

Core move: x ^ x = 0; n & (n - 1) clears the lowest set bit; shifts move bits.

Traps: Python ints are unbounded: mask with 0xFFFFFFFF and convert back for negatives (Sum of Two Integers).

```
res = 0
for x in nums: res ^= x
```

Contents

Arrays & Hashing · 9 problems

217. Contains Duplicate Easy

Problem: Return True if any value appears at least twice in the array.
Key idea: A set remembers what you've seen; a repeat hit means a duplicate.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for n in nums:
            if n in seen:
                return True
            seen.add(n)
        return False
```

242. Valid Anagram Easy

Problem: Check whether string t is a rearrangement of string s.
Key idea: Same length and identical character counts.
Complexity: O(n) time, O(1) space (26 letters)

```python
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        count = {}
        for a, b in zip(s, t):
            count[a] = count.get(a, 0) + 1
            count[b] = count.get(b, 0) - 1
        return all(v == 0 for v in count.values())
```

1. Two Sum Easy

Problem: Return indices of the two numbers that add up to target (exactly one answer exists).
Key idea: For each number, look up its complement (target - n) in a value->index map built so far.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        idx = {}
        for i, n in enumerate(nums):
            if target - n in idx:
                return [idx[target - n], i]
            idx[n] = i
```

49. Group Anagrams Medium

Problem: Group the given strings so that anagrams sit in the same group.
Key idea: Key each word by its 26-letter count tuple (or sorted string); bucket into a dict.
Complexity: O(n * k) time, O(n * k) space

```python
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        groups = defaultdict(list)
        for s in strs:
            count = [0] * 26
            for c in s:
                count[ord(c) - ord('a')] += 1
            groups[tuple(count)].append(s)
        return list(groups.values())
```

347. Top K Frequent Elements Medium

Problem: Return the k values that occur most often.
Key idea: Bucket sort: index = frequency, then walk buckets from high to low.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        count = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]
        for n, c in count.items():
            buckets[c].append(n)
        res = []
        for i in range(len(buckets) - 1, 0, -1):
            for n in buckets[i]:
                res.append(n)
                if len(res) == k:
                    return res
```

238. Product of Array Except Self Medium

Problem: For each index, output the product of every other element, without division, in O(n).
Key idea: Answer = prefix product (left of i) x suffix product (right of i); do it in two passes.
Complexity: O(n) time, O(1) extra space (output excluded)

```python
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        res = [1] * len(nums)
        prefix = 1
        for i in range(len(nums)):
            res[i] = prefix
            prefix *= nums[i]
        suffix = 1
        for i in range(len(nums) - 1, -1, -1):
            res[i] *= suffix
            suffix *= nums[i]
        return res
```

36. Valid Sudoku Medium

Problem: Decide if a partially filled 9x9 board breaks no Sudoku rule (rows, columns, 3x3 boxes).
Key idea: Three sets of sets: rows[r], cols[c], boxes[(r//3, c//3)]. Any repeat is invalid.
Complexity: O(81) = O(1) time and space

```python
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        rows, cols, boxes = defaultdict(set), defaultdict(set), defaultdict(set)
        for r in range(9):
            for c in range(9):
                v = board[r][c]
                if v == .:
                    continue
                if v in rows[r] or v in cols[c] or v in boxes[(r // 3, c // 3)]:
                    return False
                rows[r].add(v)
                cols[c].add(v)
                boxes[(r // 3, c // 3)].add(v)
        return True
```

271. Encode and Decode Strings Medium

Problem: Design encode(list of strings) -> one string and decode back, for any characters.
Key idea: Length-prefix each word: '4#leet'. Read the number up to '#', then slice that many chars.
Complexity: O(total chars) time and space

```python
class Solution:
    def encode(self, strs: List[str]) -> str:
        return .join(f{len(s)}#{s} for s in strs)

    def decode(self, s: str) -> List[str]:
        res, i = [], 0
        while i < len(s):
            j = i
            while s[j] != #:
                j += 1
            length = int(s[i:j])
            res.append(s[j + 1 : j + 1 + length])
            i = j + 1 + length
```

```
        return res
```

128. Longest Consecutive Sequence Medium

Problem: Length of the longest run of consecutive integers in an unsorted array, in O(n).
Key idea: Put all in a set; only start counting from n where n-1 is absent (a sequence start).
Complexity: O(n) time, O(n) space

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        best = 0
        for n in s:
            if n - 1 not in s:
                length = 1
                while n + length in s:
                    length += 1
                best = max(best, length)
        return best
```

Two Pointers · 5 problems

125. Valid Palindrome Easy

Problem: After keeping only alphanumerics and lower-casing, is the string a palindrome?
Key idea: Pointers from both ends, skip non-alphanumerics, compare lowercase.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l, r = 0, len(s) - 1
        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1
            if s[l].lower() != s[r].lower():
                return False
            l, r = l + 1, r - 1
        return True
```

167. Two Sum II - Input Array Is Sorted Medium

Problem: Sorted array; return 1-indexed positions of two numbers summing to target, O(1) space.
Key idea: l at start, r at end: sum too big -> r–, too small -> l++.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        l, r = 0, len(numbers) - 1
        while l < r:
            s = numbers[l] + numbers[r]
            if s > target:
                r -= 1
            elif s < target:
                l += 1
            else:
                return [l + 1, r + 1]
```

15. 3Sum Medium

Problem: Find all unique triplets that sum to zero.
Key idea: Sort; fix a, then two-pointer for the pair. Skip duplicate a's and duplicate l's.
Complexity: O(n^2) time, O(1) extra space (besides sort)

```python
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        res = []
        for i, a in enumerate(nums):
            if a > 0:
                break
            if i > 0 and a == nums[i - 1]:
                continue
            l, r = i + 1, len(nums) - 1
            while l < r:
                s = a + nums[l] + nums[r]
                if s > 0:
                    r -= 1
                elif s < 0:
                    l += 1
                else:
                    res.append([a, nums[l], nums[r]])
                    l += 1
                    while l < r and nums[l] == nums[l - 1]:
                        l += 1
        return res
```

11. Container With Most Water Medium

Problem: Pick two vertical lines that, with the x-axis, hold the most water.
Key idea: Start widest; always move the shorter wall inward (the taller one can't help).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r, best = 0, len(height) - 1, 0
        while l < r:
            best = max(best, (r - l) * min(height[l], height[r]))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return best
```

42. Trapping Rain Water Hard

Problem: Given bar heights, compute how much rain water is trapped between them.
Key idea: Water at i = min(maxLeft, maxRight) - h[i]. Two pointers: process the side with the smaller max.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0
        l, r = 0, len(height) - 1
        leftMax, rightMax = height[l], height[r]
        res = 0
        while l < r:
            if leftMax < rightMax:
                l += 1
                leftMax = max(leftMax, height[l])
                res += leftMax - height[l]
            else:
                r -= 1
                rightMax = max(rightMax, height[r])
                res += rightMax - height[r]
        return res
```

Sliding Window · 6 problems

121. Best Time to Buy and Sell Stock Easy

Problem: One buy then one later sell; maximise profit (0 if none).
Key idea: Track the minimum price so far; profit = today - min.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        lo, best = float(inf), 0
        for p in prices:
            lo = min(lo, p)
            best = max(best, p - lo)
        return best
```

3. Longest Substring Without Repeating Characters Medium

Problem: Length of the longest substring with all distinct characters.
Key idea: Window with a set; on a repeat, shrink from the left until the char is gone.
Complexity: O(n) time, O(alphabet) space

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        seen = set()
        l = best = 0
        for r in range(len(s)):
            while s[r] in seen:
                seen.remove(s[l])
                l += 1
            seen.add(s[r])
            best = max(best, r - l + 1)
        return best
```

424. Longest Repeating Character Replacement Medium

Problem: You may change up to k characters; longest substring that can become all one letter.
Key idea: Window valid while (length - count of most frequent char) <= k. maxf never needs decreasing.
Complexity: O(n) time, O(26) space

```python
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        count = {}
        l = maxf = best = 0
        for r in range(len(s)):
            count[s[r]] = count.get(s[r], 0) + 1
            maxf = max(maxf, count[s[r]])
            while (r - l + 1) - maxf > k:
                count[s[l]] -= 1
                l += 1
            best = max(best, r - l + 1)
        return best
```

567. Permutation in String Medium

Problem: Does s2 contain some rearrangement of s1 as a contiguous substring?
Key idea: Fixed-size window of len(s1); compare 26-letter counts as the window slides.
Complexity: O(n) time, O(26) space

```python
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        if len(s1) > len(s2):
            return False
        need, have = [0] * 26, [0] * 26
        for i in range(len(s1)):
            need[ord(s1[i]) - 97] += 1
            have[ord(s2[i]) - 97] += 1
        if need == have:
            return True
        for r in range(len(s1), len(s2)):
            have[ord(s2[r]) - 97] += 1
            have[ord(s2[r - len(s1)]) - 97] -= 1
            if need == have:
                return True
        return False
```

76. Minimum Window Substring Hard

Problem: Smallest substring of s containing every character of t (with multiplicity).
Key idea: Expand r until all needs met (have == need), then shrink l while still valid, recording best.
Complexity: O(|s| + |t|) time, O(alphabet) space

```python
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not t:
            return
        countT, window = Counter(t), {}
        have, need = 0, len(countT)
        res, resLen = (-1, -1), float(inf)
        l = 0
        for r, c in enumerate(s):
            window[c] = window.get(c, 0) + 1
            if c in countT and window[c] == countT[c]:
                have += 1
            while have == need:
                if r - l + 1 < resLen:
                    res, resLen = (l, r), r - l + 1
                window[s[l]] -= 1
                if s[l] in countT and window[s[l]] < countT[s[l]]:
                    have -= 1
                l += 1
        l, r = res
        return s[l : r + 1] if resLen != float(inf) else
```

239. Sliding Window Maximum Hard

Problem: Return the maximum of every window of size k.
Key idea: Monotonic decreasing deque of indices; front is the max; pop expired front.
Complexity: O(n) time, O(k) space

```python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q, res = deque(), []
        for r, n in enumerate(nums):
            while q and nums[q[-1]] < n:
                q.pop()
            q.append(r)
            if q[0] <= r - k:
                q.popleft()
            if r >= k - 1:
                res.append(nums[q[0]])
        return res
```

Stack · 7 problems

20. Valid Parentheses Easy

Problem: Are brackets ()[]{} correctly matched and nested?
Key idea: Push openers; on a closer, the top must be its matching opener.
Complexity: O(n) time, O(n) space

```
class Solution:
    def isValid(self, s: str) -> bool:
        pairs = {): (, ]: [, }: {}
        stack = []
        for c in s:
            if c in pairs:
                if not stack or stack[-1] != pairs[c]:
                    return False
                stack.pop()
            else:
                stack.append(c)
        return not stack
```

155. Min Stack Medium

Problem: Stack supporting push, pop, top and getMin, all O(1).
Key idea: Second stack storing the min at each depth.
Complexity: O(1) per op, O(n) space

```
class MinStack:
    def __init__(self):
        self.stack, self.mins = [], []

    def push(self, val: int) -> None:
        self.stack.append(val)
        self.mins.append(min(val, self.mins[-1] if self.mins else val))

    def pop(self) -> None:
        self.stack.pop()
        self.mins.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.mins[-1]
```

150. Evaluate Reverse Polish Notation Medium

Problem: Evaluate an expression in postfix notation (+ - * /, division truncates toward zero).
Key idea: Push numbers; on an operator pop b then a, push a op b. Use int(a / b) for truncation.
Complexity: O(n) time, O(n) space

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        st = []
        for t in tokens:
            if t in +-*/:
                b, a = st.pop(), st.pop()
                if t == +: st.append(a + b)
                elif t == -: st.append(a - b)
                elif t == *: st.append(a * b)
                else: st.append(int(a / b))
            else:
                st.append(int(t))
        return st[0]
```

22. Generate Parentheses Medium

Problem: List every well-formed string of n pairs of parentheses.
Key idea: Backtrack: add '(' if open $<$ n; add ')' if close $<$ open.
Complexity: O(4^n / sqrt(n)) time (Catalan), O(n) recursion

```python
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        res, cur = [], []
        def bt(op, cl):
            if op == cl == n:
                res.append(.join(cur))
                return
            if op < n:
                cur.append((); bt(op + 1, cl); cur.pop()
            if cl < op:
                cur.append()); bt(op, cl + 1); cur.pop()
        bt(0, 0)
        return res
```

739. Daily Temperatures Medium

Problem: For each day, how many days until a warmer temperature (0 if never)?
Key idea: Monotonic decreasing stack of indices; a warmer day resolves everything colder on the stack.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        res = [0] * len(temperatures)
        st = []  # indices
        for i, t in enumerate(temperatures):
            while st and temperatures[st[-1]] < t:
                j = st.pop()
                res[j] = i - j
            st.append(i)
        return res
```

853. Car Fleet Medium

Problem: Cars drive to target; a faster car that catches a slower one joins it. Count arriving fleets.
Key idea: Sort by position descending; compute arrival time. A car forms a new fleet only if it takes longer than the fleet ahead.
Complexity: O(n log n) time, O(n) space

```python
class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        cars = sorted(zip(position, speed), reverse=True)
        st = []
        for p, s in cars:
            t = (target - p) / s
            if not st or t > st[-1]:
                st.append(t)
        return len(st)
```

84. Largest Rectangle in Histogram Hard

Problem: Area of the largest rectangle that fits under the histogram bars.
Key idea: Increasing stack of (start, height). When popping a taller bar, its rectangle ends here; the new bar inherits the popped start.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        best, st = 0, []  # (start index, height)
        for i, h in enumerate(heights):
            start = i
            while st and st[-1][1] > h:
                idx, height = st.pop()
                best = max(best, height * (i - idx))
                start = idx
            st.append((start, h))
        for idx, height in st:
            best = max(best, height * (len(heights) - idx))
        return best
```

Binary Search · 7 problems

704. Binary Search Easy

Problem: Return the index of target in a sorted array, or -1.
Key idea: Classic l <= r loop with mid = l + (r - l) // 2.
Complexity: O(log n) time, O(1) space

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[m] < target:
                l = m + 1
            else:
                r = m - 1
        return -1
```

74. Search a 2D Matrix Medium

Problem: Rows sorted and each row starts after the previous ends. Is target present?
Key idea: Treat it as one sorted array of m*n: index -> (i // n, i % n).
Complexity: O(log(m*n)) time, O(1) space

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        l, r = 0, m * n - 1
        while l <= r:
            mid = (l + r) // 2
            v = matrix[mid // n][mid % n]
            if v == target:
                return True
            if v < target:
                l = mid + 1
            else:
                r = mid - 1
        return False
```

875. Koko Eating Bananas Medium

Problem: Minimum eating speed k so all piles are finished within h hours (one pile per hour max).
Key idea: Binary search on the answer k in [1, max(piles)]; hours(k) = sum(ceil(p / k)).
Complexity: O(n log max) time, O(1) space

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        l, r = 1, max(piles)
        res = r
        while l <= r:
            k = (l + r) // 2
            hours = sum((p + k - 1) // k for p in piles)
            if hours <= h:
                res = k
                r = k - 1
            else:
                l = k + 1
        return res
```

153. Find Minimum in Rotated Sorted Array Medium

Problem: A sorted array of distinct values was rotated; find its minimum in O(log n).
Key idea: Compare mid with the right end: nums[m] > nums[r] means the min is right of m.
Complexity: O(log n) time, O(1) space

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            m = (l + r) // 2
            if nums[m] > nums[r]:
                l = m + 1
            else:
                r = m
```

```
        return nums[l]
```

33. Search in Rotated Sorted Array Medium

Problem: Find target's index in a rotated sorted array of distinct values, O(log n).
Key idea: One half is always sorted. Check if target lies inside the sorted half; else go to the other half.
Complexity: O(log n) time, O(1) space

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]:            # left half sorted
                if nums[l] <= target < nums[m]:
                    r = m - 1
                else:
                    l = m + 1
            else:                              # right half sorted
                if nums[m] < target <= nums[r]:
                    l = m + 1
                else:
                    r = m - 1
        return -1
```

981. Time Based Key-Value Store Medium

Problem: set(key, value, timestamp); get(key, t) returns the value with the largest timestamp <= t.
Key idea: Timestamps arrive increasing, so each key's list is sorted: binary search for the last ts <= t.
Complexity: set O(1), get O(log n); O(n) space

```
class TimeMap:
    def __init__(self):
        self.store = defaultdict(list)  # key -> [(ts, value)]

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        arr, res = self.store[key],
        l, r = 0, len(arr) - 1
        while l <= r:
            m = (l + r) // 2
            if arr[m][0] <= timestamp:
                res = arr[m][1]
                l = m + 1
            else:
                r = m - 1
        return res
```

4. Median of Two Sorted Arrays Hard

Problem: Median of two sorted arrays combined, in O(log(min(m, n))).
Key idea: Binary search a cut in the shorter array so left halves together hold half the elements and Aleft <= Bright, Bleft <= Aright.
Complexity: O(log min(m, n)) time, O(1) space

```
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        A, B = nums1, nums2
        if len(A) > len(B):
            A, B = B, A
        total = len(A) + len(B)
        half = total // 2
        l, r = 0, len(A)
        while True:
            i = (l + r) // 2          # elements taken from A
            j = half - i              # elements taken from B
            Aleft = A[i - 1] if i > 0 else float(-inf)
            Aright = A[i] if i < len(A) else float(inf)
            Bleft = B[j - 1] if j > 0 else float(-inf)
            Bright = B[j] if j < len(B) else float(inf)
            if Aleft <= Bright and Bleft <= Aright:
                if total % 2:
                    return min(Aright, Bright)
                return (max(Aleft, Bleft) + min(Aright, Bright)) / 2
```

```
            if Aleft > Bright:
                r = i - 1
            else:
                l = i + 1
```

Linked List · 11 problems

206. Reverse Linked List Easy

Problem: Reverse a singly linked list.
Key idea: Walk once, pointing each node back to prev.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev, cur = None, head
        while cur:
            nxt = cur.next
            cur.next = prev
            prev, cur = cur, nxt
        return prev
```

21. Merge Two Sorted Lists Easy

Problem: Splice two sorted lists into one sorted list.
Key idea: Dummy head; attach the smaller node each step; append the leftover.
Complexity: O(m + n) time, O(1) space

```python
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = tail = ListNode()
        while list1 and list2:
            if list1.val <= list2.val:
                tail.next, list1 = list1, list1.next
            else:
                tail.next, list2 = list2, list2.next
            tail = tail.next
        tail.next = list1 or list2
        return dummy.next
```

143. Reorder List Medium

Problem: Rearrange L0,L1,...,Ln into L0,Ln,L1,Ln-1,... in place.
Key idea: Three steps: find middle (slow/fast), reverse the second half, merge alternately.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        second = slow.next
        slow.next = None
        prev = None
        while second:                       # reverse second half
            nxt = second.next
            second.next = prev
            prev, second = second, nxt
        first, second = head, prev
        while second:                       # merge
            t1, t2 = first.next, second.next
            first.next = second
            second.next = t1
            first, second = t1, t2
```

19. Remove Nth Node From End of List Medium

Problem: Delete the n-th node from the end in one pass.
Key idea: Dummy head; move right pointer n ahead, then move both until right is None; left.next is the target.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        left, right = dummy, head
        for _ in range(n):
            right = right.next
        while right:
            left, right = left.next, right.next
        left.next = left.next.next
        return dummy.next
```

138. Copy List with Random Pointer Medium

Problem: Deep-copy a list where each node also has a random pointer.
Key idea: Pass 1: map old -> new node. Pass 2: wire next and random through the map.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def copyRandomList(self, head: Optional[Node]) -> Optional[Node]:
        old_to_new = {None: None}
        cur = head
        while cur:
            old_to_new[cur] = Node(cur.val)
            cur = cur.next
        cur = head
        while cur:
            copy = old_to_new[cur]
            copy.next = old_to_new[cur.next]
            copy.random = old_to_new[cur.random]
            cur = cur.next
        return old_to_new[head]
```

2. Add Two Numbers Medium

Problem: Two numbers stored as reversed digit lists; return their sum as a list.
Key idea: Add digit by digit with a carry; keep going while either list or carry remains.
Complexity: O(max(m, n)) time, O(1) extra space

```python
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = cur = ListNode()
        carry = 0
        while l1 or l2 or carry:
            v = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, v = divmod(v, 10)
            cur.next = ListNode(v)
            cur = cur.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next
```

141. Linked List Cycle Easy

Problem: Does the list contain a cycle?
Key idea: Floyd's tortoise and hare: if fast ever meets slow, there is a cycle.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            if slow is fast:
                return True
        return False
```

287. Find the Duplicate Number Medium

Problem: Array of n+1 ints in [1, n] has one repeated value; find it without modifying the array, O(1) space.
Key idea: Treat i -> nums[i] as a linked list; the duplicate is the cycle entrance (Floyd, phase 2).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        slow = fast = 0
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        slow2 = 0
        while slow != slow2:
            slow, slow2 = nums[slow], nums[slow2]
        return slow
```

146. LRU Cache Medium

Problem: Fixed-capacity cache with O(1) get/put that evicts the least recently used key.
Key idea: Hash map key -> node, plus a doubly linked list ordered by recency (LRU at left, MRU at right).
Complexity: O(1) per op, O(capacity) space

```
class DNode:
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.cap, self.cache = capacity, {}
        self.left, self.right = DNode(), DNode()      # left = LRU, right = MRU
        self.left.next, self.right.prev = self.right, self.left

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _insert(self, node):                          # insert before right
        prev = self.right.prev
        prev.next = node
        node.prev, node.next = prev, self.right
        self.right.prev = node

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._remove(node)
        self._insert(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self._remove(self.cache[key])
        self.cache[key] = DNode(key, value)
        self._insert(self.cache[key])
        if len(self.cache) > self.cap:
            lru = self.left.next
            self._remove(lru)
            del self.cache[lru.key]
```

23. Merge k Sorted Lists Hard

Problem: Merge k sorted linked lists into one sorted list.
Key idea: Min-heap of (value, list index, node); pop the smallest and push its next.
Complexity: O(N log k) time, O(k) space

```
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        heap = [(node.val, i, node) for i, node in enumerate(lists) if node]
        heapq.heapify(heap)
        dummy = tail = ListNode()
        while heap:
            _, i, node = heapq.heappop(heap)
            tail.next = node
            tail = node
            if node.next:
                heapq.heappush(heap, (node.next.val, i, node.next))
        return dummy.next
```

25. Reverse Nodes in k-Group Hard

Problem: Reverse the list k nodes at a time; a leftover tail shorter than k stays as is.
Key idea: For each group: find the k-th node; reverse the group; reconnect the previous group's tail.
Complexity: O(n) time, O(1) space

```
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        group_prev = dummy
        while True:
            kth = group_prev
            for _ in range(k):
                kth = kth.next
                if not kth:
```

```
            return dummy.next
        group_next = kth.next
        prev, cur = group_next, group_prev.next
        while cur is not group_next:
            nxt = cur.next
            cur.next = prev
            prev, cur = cur, nxt
        first = group_prev.next          # becomes the group's tail
        group_prev.next = kth
        group_prev = first
```

Trees · 15 problems

226. Invert Binary Tree Easy

Problem: Mirror the tree (swap every left and right child).
Key idea: Swap children, recurse on both.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root:
            root.left, root.right = self.invertTree(root.right), self.invertTree(root.left)
        return root
```

104. Maximum Depth of Binary Tree Easy

Problem: Number of nodes on the longest root-to-leaf path.
Key idea: depth = 1 + max(depth(left), depth(right)).
Complexity: O(n) time, O(h) space

```python
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

543. Diameter of Binary Tree Easy

Problem: Longest path (in edges) between any two nodes.
Key idea: DFS returns height; at each node update best with left height + right height.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        best = 0
        def height(node):
            nonlocal best
            if not node:
                return 0
            l, r = height(node.left), height(node.right)
            best = max(best, l + r)
            return 1 + max(l, r)
        height(root)
        return best
```

110. Balanced Binary Tree Easy

Problem: Is every node's left/right height difference at most 1?
Key idea: Bottom-up DFS returning height, or -1 as an 'unbalanced' sentinel.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def h(node):
            if not node:
                return 0
            l, r = h(node.left), h(node.right)
            if l < 0 or r < 0 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)
        return h(root) >= 0
```

100. Same Tree Easy

Problem: Are two trees identical in structure and values?
Key idea: Both None -> True; one None or values differ -> False; else recurse on both sides.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if not p and not q:
            return True
        if not p or not q or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

572. Subtree of Another Tree Easy

Problem: Is subRoot an exact subtree somewhere inside root?
Key idea: At every node of root, test sameTree(node, subRoot).
Complexity: O(m * n) time, O(h) space

```python
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        def same(a, b):
            if not a and not b:
                return True
            if not a or not b or a.val != b.val:
                return False
            return same(a.left, b.left) and same(a.right, b.right)
        if not subRoot:
            return True
        if not root:
            return False
        return (same(root, subRoot) or self.isSubtree(root.left, subRoot)
                or self.isSubtree(root.right, subRoot))
```

235. Lowest Common Ancestor of a Binary Search Tree Medium

Problem: Find the LCA of nodes p and q in a BST.
Key idea: Both smaller -> go left; both larger -> go right; otherwise this node is where they split.
Complexity: O(h) time, O(1) space

```python
class Solution:
    def lowestCommonAncestor(self, root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
        cur = root
        while cur:
            if p.val < cur.val and q.val < cur.val:
                cur = cur.left
            elif p.val > cur.val and q.val > cur.val:
                cur = cur.right
            else:
                return cur
```

102. Binary Tree Level Order Traversal Medium

Problem: Return node values level by level.
Key idea: BFS with a queue; process len(queue) nodes per level.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        res, q = [], deque([root] if root else [])
        while q:
            level = []
            for _ in range(len(q)):
                node = q.popleft()
                level.append(node.val)
                if node.left: q.append(node.left)
                if node.right: q.append(node.right)
            res.append(level)
        return res
```

199. Binary Tree Right Side View Medium

Problem: Values visible when looking at the tree from the right, top to bottom.
Key idea: BFS by level; take the last node of each level.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        res, q = [], deque([root] if root else [])
        while q:
            for i in range(len(q)):
                node = q.popleft()
                if i == 0:
                    res.append(node.val)       # enqueue right first -> first is rightmost
                if node.right: q.append(node.right)
                if node.left: q.append(node.left)
        return res
```

1448. Count Good Nodes in Binary Tree Medium

Problem: A node is good if no node on the path from the root to it is larger. Count good nodes.
Key idea: DFS carrying the max value seen on the path.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(node, mx):
            if not node:
                return 0
            good = 1 if node.val >= mx else 0
            mx = max(mx, node.val)
            return good + dfs(node.left, mx) + dfs(node.right, mx)
        return dfs(root, root.val)
```

98. Validate Binary Search Tree Medium

Problem: Is the tree a valid BST (strict ordering)?
Key idea: DFS passing an open interval (low, high) that every node must sit inside.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def ok(node, lo, hi):
            if not node:
                return True
            if not (lo < node.val < hi):
                return False
            return ok(node.left, lo, node.val) and ok(node.right, node.val, hi)
        return ok(root, float(-inf), float(inf))
```

230. Kth Smallest Element in a BST Medium

Problem: Return the k-th smallest value (1-indexed) in a BST.
Key idea: In-order traversal visits values in sorted order; stop at the k-th pop.
Complexity: O(h + k) time, O(h) space

```python
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        st, cur = [], root
        while st or cur:
            while cur:
                st.append(cur)
                cur = cur.left
            cur = st.pop()
            k -= 1
            if k == 0:
                return cur.val
            cur = cur.right
```

105. Construct Binary Tree from Preorder and Inorder Traversal Medium

Problem: Rebuild the tree from its preorder and inorder sequences (unique values).
Key idea: preorder[0] is the root; its index in inorder splits left/right subtrees. Use a value->index map.
Complexity: O(n) time, O(n) space

```python
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        idx = {v: i for i, v in enumerate(inorder)}
        pre = iter(preorder)
        def build(l, r):
            if l > r:
                return None
            val = next(pre)
            node = TreeNode(val)
            m = idx[val]
            node.left = build(l, m - 1)
            node.right = build(m + 1, r)
            return node
        return build(0, len(inorder) - 1)
```

124. Binary Tree Maximum Path Sum Hard

Problem: Max sum of any path (any start and end node, must be connected).
Key idea: DFS returns the best downward 'arm' (clamped at 0); at each node update answer with val + left + right.
Complexity: O(n) time, O(h) space

```python
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        best = root.val
        def arm(node):
            nonlocal best
            if not node:
                return 0
            l = max(arm(node.left), 0)
            r = max(arm(node.right), 0)
            best = max(best, node.val + l + r)
            return node.val + max(l, r)
        arm(root)
        return best
```

297. Serialize and Deserialize Binary Tree Hard

Problem: Convert a tree to a string and back.
Key idea: Preorder with 'N' for nulls; deserialize by consuming tokens in the same preorder.
Complexity: O(n) time, O(n) space

```python
class Codec:
    def serialize(self, root):
        out = []
        def dfs(node):
            if not node:
                out.append(N)
                return
            out.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ,.join(out)

    def deserialize(self, data):
        vals = iter(data.split(,))
        def dfs():
            v = next(vals)
            if v == N:
                return None
            node = TreeNode(int(v))
            node.left = dfs()
            node.right = dfs()
            return node
        return dfs()
```

Tries · 3 problems

208. Implement Trie (Prefix Tree) Medium

Problem: Build insert, search (whole word) and startsWith (prefix).
Key idea: Each node: dict of children + end-of-word flag.
Complexity: O(L) per op, O(total chars) space

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        cur = self.root
        for c in word:
            cur = cur.children.setdefault(c, TrieNode())
        cur.end = True

    def _walk(self, s):
        cur = self.root
        for c in s:
            if c not in cur.children:
                return None
            cur = cur.children[c]
        return cur

    def search(self, word: str) -> bool:
        node = self._walk(word)
        return bool(node and node.end)

    def startsWith(self, prefix: str) -> bool:
        return self._walk(prefix) is not None
```

211. Design Add and Search Words Data Structure Medium

Problem: Like a trie, but search may contain '.' matching any one letter.
Key idea: Trie + DFS; on '.', try every child.
Complexity: add O(L); search O(26^dots * L) worst case

```python
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        cur = self.root
        for c in word:
            cur = cur.children.setdefault(c, TrieNode())
        cur.end = True

    def search(self, word: str) -> bool:
        def dfs(i, node):
            if i == len(word):
                return node.end
            c = word[i]
            if c == .:
                return any(dfs(i + 1, ch) for ch in node.children.values())
            return c in node.children and dfs(i + 1, node.children[c])
        return dfs(0, self.root)
```

212. Word Search II Hard

Problem: Find all dictionary words that can be traced on a letter grid (adjacent cells, no reuse).
Key idea: Insert words into a trie; DFS from each cell following trie edges; prune found words.
Complexity: O(m * n * 4^L) worst case, O(total word chars) space

```
class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        root = {}
        for w in words:
            node = root
            for c in w:
                node = node.setdefault(c, {})
            node[$] = w
        R, C = len(board), len(board[0])
        res = []
        def dfs(r, c, parent):
            ch = board[r][c]
            node = parent[ch]
            if $ in node:
                res.append(node.pop($))     # avoid duplicates
            board[r][c] = #
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < R and 0 <= nc < C and board[nr][nc] in node:
                    dfs(nr, nc, node)
            board[r][c] = ch
            if not node:                        # prune empty branch
                parent.pop(ch)
        for r in range(R):
            for c in range(C):
                if board[r][c] in root:
                    dfs(r, c, root)
        return res
```

Heap / Priority Queue · 7 problems

703. Kth Largest Element in a Stream Easy

Problem: Class that returns the k-th largest after each added value.
Key idea: Keep a min-heap of size k; its top is the k-th largest.
Complexity: O(log k) per add, O(k) space

```python
class KthLargest:
    def __init__(self, k: int, nums: List[int]):
        self.k, self.heap = k, nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val: int) -> int:
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

1046. Last Stone Weight Easy

Problem: Repeatedly smash the two heaviest stones; return the last stone's weight (or 0).
Key idea: Max-heap via negated values.
Complexity: O(n log n) time, O(n) space

```python
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-s for s in stones]
        heapq.heapify(h)
        while len(h) > 1:
            a, b = -heapq.heappop(h), -heapq.heappop(h)
            if a != b:
                heapq.heappush(h, -(a - b))
        return -h[0] if h else 0
```

973. K Closest Points to Origin Medium

Problem: Return the k points nearest to (0, 0).
Key idea: Max-heap of size k on squared distance (negated); drop the farthest when over k.
Complexity: O(n log k) time, O(k) space

```python
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        h = []
        for x, y in points:
            heapq.heappush(h, (-(x * x + y * y), x, y))
            if len(h) > k:
                heapq.heappop(h)
        return [[x, y] for _, x, y in h]
```

215. Kth Largest Element in an Array Medium

Problem: Return the k-th largest value in an unsorted array.
Key idea: Min-heap of size k (or Quickselect for average O(n)).
Complexity: O(n log k) time, O(k) space

```python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        h = []
        for n in nums:
            heapq.heappush(h, n)
            if len(h) > k:
                heapq.heappop(h)
        return h[0]
```

621. Task Scheduler Medium

Problem: Same tasks need n idle slots between them; minimum total time units to run all.
Key idea: Formula: (maxCount - 1) * (n + 1) + numberOfTasksWithMaxCount, but at least len(tasks).
Complexity: O(N) time, O(26) space

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        count = Counter(tasks)
        mx = max(count.values())
        num_mx = sum(1 for v in count.values() if v == mx)
        return max(len(tasks), (mx - 1) * (n + 1) + num_mx)
```

355. Design Twitter Medium

Problem: postTweet, follow, unfollow, getNewsFeed (10 most recent from self + followees).
Key idea: Global decreasing time counter; merge the followees' tweet lists with a heap (k-way merge).
Complexity: getNewsFeed O(F log F) for F followees; O(users + tweets) space

```
class Twitter:
    def __init__(self):
        self.time = 0
        self.tweets = defaultdict(list)    # user -> [(time, tweetId)]
        self.following = defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].append((self.time, tweetId))
        self.time -= 1                     # smaller = newer (min-heap)

    def getNewsFeed(self, userId: int) -> List[int]:
        res, h = [], []
        users = self.following[userId] | {userId}
        for u in users:
            if self.tweets[u]:
                i = len(self.tweets[u]) - 1
                t, tid = self.tweets[u][i]
                h.append((t, tid, u, i - 1))
        heapq.heapify(h)
        while h and len(res) < 10:
            t, tid, u, i = heapq.heappop(h)
            res.append(tid)
            if i >= 0:
                t2, tid2 = self.tweets[u][i]
                heapq.heappush(h, (t2, tid2, u, i - 1))
        return res

    def follow(self, followerId: int, followeeId: int) -> None:
        if followerId != followeeId:
            self.following[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.following[followerId].discard(followeeId)
```

295. Find Median from Data Stream Hard

Problem: Support addNum and findMedian on a growing stream.
Key idea: Max-heap for the lower half, min-heap for the upper half; keep sizes within 1.
Complexity: addNum O(log n), findMedian O(1)

```
class MedianFinder:
    def __init__(self):
        self.small, self.large = [], []    # small = max-heap (negated)

    def addNum(self, num: int) -> None:
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2
```

Backtracking · 9 problems

78. Subsets Medium

Problem: Return all subsets of distinct integers.
Key idea: At each index choose: include nums[i] or skip it.
Complexity: O(n * 2^n) time, O(n) recursion

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res, cur = [], []
        def dfs(i):
            if i == len(nums):
                res.append(cur[:])
                return
            cur.append(nums[i]); dfs(i + 1); cur.pop()
            dfs(i + 1)
        dfs(0)
        return res
```

39. Combination Sum Medium

Problem: Unique combinations of candidates summing to target; each candidate may be reused.
Key idea: Branch: use candidates[i] again (stay at i) or move to i+1. Stop when total >= target.
Complexity: O(2^(t/min)) time, O(t/min) recursion

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        res, cur = [], []
        def dfs(i, total):
            if total == target:
                res.append(cur[:])
                return
            if i == len(candidates) or total > target:
                return
            cur.append(candidates[i]); dfs(i, total + candidates[i]); cur.pop()
            dfs(i + 1, total)
        dfs(0, 0)
        return res
```

46. Permutations Medium

Problem: Return all orderings of distinct integers.
Key idea: Build position by position, using each unused number once.
Complexity: O(n * n!) time, O(n) recursion

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res, cur, used = [], [], [False] * len(nums)
        def dfs():
            if len(cur) == len(nums):
                res.append(cur[:])
                return
            for i, n in enumerate(nums):
                if not used[i]:
                    used[i] = True
                    cur.append(n); dfs(); cur.pop()
                    used[i] = False
        dfs()
        return res
```

90. Subsets II Medium

Problem: All unique subsets when the input may contain duplicates.
Key idea: Sort; after choosing to skip nums[i], also skip all equal values.
Complexity: O(n * 2^n) time, O(n) recursion

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        res, cur = [], []
        def dfs(i):
            if i == len(nums):
                res.append(cur[:])
                return
            cur.append(nums[i]); dfs(i + 1); cur.pop()
            while i + 1 < len(nums) and nums[i] == nums[i + 1]:
```

```
                i += 1
            dfs(i + 1)
        dfs(0)
        return res
```

40. Combination Sum II Medium

Problem: Each candidate used at most once, input has duplicates; unique combos summing to target.
Key idea: Sort; loop over choices from i, skipping j > i where c[j] == c[j-1]; break when too big.
Complexity: O(2^n) time, O(n) recursion

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        candidates.sort()
        res, cur = [], []
        def dfs(i, remain):
            if remain == 0:
                res.append(cur[:])
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                if candidates[j] > remain:
                    break
                cur.append(candidates[j]); dfs(j + 1, remain - candidates[j]); cur.pop()
        dfs(0, target)
        return res
```

79. Word Search Medium

Problem: Can word be traced through adjacent grid cells, each cell used at most once?
Key idea: DFS from each cell; mark visited in place, restore on backtrack.
Complexity: O(m * n * 4^L) time, O(L) recursion

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        R, C = len(board), len(board[0])
        def dfs(r, c, i):
            if i == len(word):
                return True
            if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[i]:
                return False
            tmp, board[r][c] = board[r][c], #
            found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                     dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
            board[r][c] = tmp
            return found
        return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

131. Palindrome Partitioning Medium

Problem: Split s every possible way so that every piece is a palindrome.
Key idea: From index i, try every end j where s[i:j+1] is a palindrome; recurse from j+1.
Complexity: O(n * 2^n) time, O(n) recursion

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        res, cur = [], []
        def dfs(i):
            if i == len(s):
                res.append(cur[:])
                return
            for j in range(i, len(s)):
                piece = s[i : j + 1]
                if piece == piece[::-1]:
                    cur.append(piece); dfs(j + 1); cur.pop()
        dfs(0)
        return res
```

17. Letter Combinations of a Phone Number Medium

Problem: All letter strings a digit string (2-9) could represent on a phone keypad.
Key idea: Backtrack over digits, appending each mapped letter.
Complexity: O(n * 4^n) time, O(n) recursion

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        keys = {2: abc, 3: def, 4: ghi, 5: jkl,
                6: mno, 7: pqrs, 8: tuv, 9: wxyz}
        res = []
        def dfs(i, cur):
            if i == len(digits):
                res.append(cur)
                return
            for ch in keys[digits[i]]:
                dfs(i + 1, cur + ch)
        dfs(0, )
        return res
```

51. N-Queens Hard

Problem: All ways to place n queens on an n x n board with no two attacking.
Key idea: Row by row; track used columns, (r + c) diagonals and (r - c) anti-diagonals in sets.
Complexity: O(n!) time, O(n^2) for board

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        cols, pos, neg = set(), set(), set()
        board = [[.] * n for _ in range(n)]
        res = []
        def dfs(r):
            if r == n:
                res.append([.join(row) for row in board])
                return
            for c in range(n):
                if c in cols or (r + c) in pos or (r - c) in neg:
                    continue
                cols.add(c); pos.add(r + c); neg.add(r - c); board[r][c] = Q
                dfs(r + 1)
                cols.remove(c); pos.remove(r + c); neg.remove(r - c); board[r][c] = .
        dfs(0)
        return res
```

Graphs · 13 problems

200. Number of Islands Medium

Problem: Count groups of connected '1's (4-directional) in a grid.
Key idea: Scan; at each unvisited land cell, count +1 and flood-fill (DFS/BFS) the whole island.
Complexity: O(m * n) time and space

```python
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        R, C = len(grid), len(grid[0])
        def sink(r, c):
            if 0 <= r < R and 0 <= c < C and grid[r][c] == 1:
                grid[r][c] = 0
                sink(r + 1, c); sink(r - 1, c); sink(r, c + 1); sink(r, c - 1)
        count = 0
        for r in range(R):
            for c in range(C):
                if grid[r][c] == 1:
                    count += 1
                    sink(r, c)
        return count
```

133. Clone Graph Medium

Problem: Deep-copy a connected undirected graph given one node.
Key idea: DFS with a map old -> clone; create the clone before visiting neighbours (handles cycles).
Complexity: O(V + E) time, O(V) space

```python
class Solution:
    def cloneGraph(self, node: Optional[Node]) -> Optional[Node]:
        clones = {}
        def dfs(n):
            if n in clones:
                return clones[n]
            copy = Node(n.val)
            clones[n] = copy
            for nb in n.neighbors:
                copy.neighbors.append(dfs(nb))
            return copy
        return dfs(node) if node else None
```

695. Max Area of Island Medium

Problem: Size of the largest island of 1s.
Key idea: Flood-fill returning the area; take the max.
Complexity: O(m * n) time and space

```python
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        R, C = len(grid), len(grid[0])
        def area(r, c):
            if not (0 <= r < R and 0 <= c < C) or grid[r][c] != 1:
                return 0
            grid[r][c] = 0
            return 1 + area(r + 1, c) + area(r - 1, c) + area(r, c + 1) + area(r, c - 1)
        return max(area(r, c) for r in range(R) for c in range(C))
```

417. Pacific Atlantic Water Flow Medium

Problem: Cells whose water can flow (to equal or lower neighbours) to both the Pacific (top/left) and Atlantic (bottom/right).
Key idea: Reverse it: DFS uphill from each ocean's border; answer = intersection of both reachable sets.
Complexity: O(m * n) time and space

```python
class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        R, C = len(heights), len(heights[0])
        pac, atl = set(), set()
        def dfs(r, c, seen, prev):
            if ((r, c) in seen or not (0 <= r < R and 0 <= c < C)
                    or heights[r][c] < prev):
                return
            seen.add((r, c))
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                dfs(r + dr, c + dc, seen, heights[r][c])
        for c in range(C):
            dfs(0, c, pac, heights[0][c])
            dfs(R - 1, c, atl, heights[R - 1][c])
```

```
        for r in range(R):
            dfs(r, 0, pac, heights[r][0])
            dfs(r, C - 1, atl, heights[r][C - 1])
        return [list(p) for p in pac & atl]
```

130. Surrounded Regions Medium

Problem: Flip every 'O' region not connected to the border into 'X'.
Key idea: Mark border-connected O's as safe (T) by DFS; flip the rest to X; restore T to O.
Complexity: O(m * n) time and space

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        R, C = len(board), len(board[0])
        def mark(r, c):
            if 0 <= r < R and 0 <= c < C and board[r][c] == O:
                board[r][c] = T
                mark(r + 1, c); mark(r - 1, c); mark(r, c + 1); mark(r, c - 1)
        for r in range(R):
            mark(r, 0); mark(r, C - 1)
        for c in range(C):
            mark(0, c); mark(R - 1, c)
        for r in range(R):
            for c in range(C):
                if board[r][c] == O:
                    board[r][c] = X
                elif board[r][c] == T:
                    board[r][c] = O
```

994. Rotting Oranges Medium

Problem: Each minute rot spreads to adjacent fresh oranges. Minutes until none are fresh (-1 if impossible).
Key idea: Multi-source BFS from all rotten oranges at once; count levels.
Complexity: O(m * n) time and space

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        R, C = len(grid), len(grid[0])
        q, fresh = deque(), 0
        for r in range(R):
            for c in range(C):
                if grid[r][c] == 2: q.append((r, c))
                elif grid[r][c] == 1: fresh += 1
        minutes = 0
        while q and fresh:
            for _ in range(len(q)):
                r, c = q.popleft()
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                    if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        q.append((nr, nc))
            minutes += 1
        return minutes if fresh == 0 else -1
```

286. Walls and Gates Medium

Problem: Fill each empty room (INF) with its distance to the nearest gate (0); walls are -1.
Key idea: Multi-source BFS starting from every gate.
Complexity: O(m * n) time and space

```
class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        INF = 2147483647
        R, C = len(rooms), len(rooms[0])
        q = deque((r, c) for r in range(R) for c in range(C) if rooms[r][c] == 0)
        while q:
            r, c = q.popleft()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < R and 0 <= nc < C and rooms[nr][nc] == INF:
                    rooms[nr][nc] = rooms[r][c] + 1
                    q.append((nr, nc))
```

207. Course Schedule Medium

Problem: Given prerequisite pairs, can all courses be finished (i.e. no cycle)?
Key idea: Kahn's topological sort: repeatedly take in-degree-0 nodes; success if all are taken.
Complexity: O(V + E) time and space

```python
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        adj = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:          # b -> a
            adj[b].append(a)
            indeg[a] += 1
        q = deque(i for i in range(numCourses) if indeg[i] == 0)
        taken = 0
        while q:
            c = q.popleft()
            taken += 1
            for nxt in adj[c]:
                indeg[nxt] -= 1
                if indeg[nxt] == 0:
                    q.append(nxt)
        return taken == numCourses
```

210. Course Schedule II Medium

Problem: Return a valid order to take all courses, or [] if impossible.
Key idea: Same Kahn's algorithm; the pop order is the answer.
Complexity: O(V + E) time and space

```python
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        adj = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            adj[b].append(a)
            indeg[a] += 1
        q = deque(i for i in range(numCourses) if indeg[i] == 0)
        order = []
        while q:
            c = q.popleft()
            order.append(c)
            for nxt in adj[c]:
                indeg[nxt] -= 1
                if indeg[nxt] == 0:
                    q.append(nxt)
        return order if len(order) == numCourses else []
```

684. Redundant Connection Medium

Problem: A tree plus one extra edge; return the edge (latest in input) that creates the cycle.
Key idea: Union-Find: the first edge whose endpoints already share a root is redundant.
Complexity: O(n * alpha(n)) time, O(n) space

```python
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        parent = list(range(len(edges) + 1))
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]   # path compression
                x = parent[x]
            return x
        for a, b in edges:
            ra, rb = find(a), find(b)
            if ra == rb:
                return [a, b]
            parent[ra] = rb
```

323. Number of Connected Components in an Undirected Graph Medium

Problem: n nodes and an edge list; count connected components.
Key idea: Union-Find: start with n components, subtract 1 for every successful union.
Complexity: O(E * alpha(n)) time, O(n) space

```
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        parent = list(range(n))
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        comps = n
        for a, b in edges:
            ra, rb = find(a), find(b)
            if ra != rb:
                parent[ra] = rb
                comps -= 1
        return comps
```

261. Graph Valid Tree Medium

Problem: Do n nodes and the given undirected edges form a valid tree?
Key idea: A tree has exactly n-1 edges and no cycle (equivalently: n-1 edges and connected).
Complexity: O(n * alpha(n)) time, O(n) space

```
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        if len(edges) != n - 1:
            return False
        parent = list(range(n))
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        for a, b in edges:
            ra, rb = find(a), find(b)
            if ra == rb:
                return False
            parent[ra] = rb
        return True
```

127. Word Ladder Hard

Problem: Fewest words in a transformation from beginWord to endWord, changing one letter at a time via wordList.
Key idea: BFS; neighbours found via wildcard patterns like 'h*t' -> words.
Complexity: O(N * L^2) time, O(N * L^2) space

```
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) -> int:
        if endWord not in wordList:
            return 0
        patterns = defaultdict(list)
        for w in wordList + [beginWord]:
            for i in range(len(w)):
                patterns[w[:i] + * + w[i + 1:]].append(w)
        seen, q, steps = {beginWord}, deque([beginWord]), 1
        while q:
            for _ in range(len(q)):
                w = q.popleft()
                if w == endWord:
                    return steps
                for i in range(len(w)):
                    for nb in patterns[w[:i] + * + w[i + 1:]]:
                        if nb not in seen:
                            seen.add(nb)
                            q.append(nb)
            steps += 1
        return 0
```

Advanced Graphs · 6 problems

332. Reconstruct Itinerary Hard

Problem: Use every ticket exactly once starting at JFK; return the lexically smallest valid route.
Key idea: Hierholzer's algorithm: adjacency lists sorted in reverse (pop smallest), DFS, append on backtrack, reverse at end.
Complexity: O(E log E) time, O(E) space

```python
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        adj = defaultdict(list)
        for src, dst in sorted(tickets, reverse=True):
            adj[src].append(dst)            # pop() gives smallest
        route, st = [], [JFK]
        while st:
            while adj[st[-1]]:
                st.append(adj[st[-1]].pop())
            route.append(st.pop())
        return route[::-1]
```

1584. Min Cost to Connect All Points Medium

Problem: Connect all points with minimum total Manhattan-distance edges.
Key idea: Prim's MST with a min-heap starting from point 0.
Complexity: O(n^2 log n) time, O(n^2) space

```python
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        seen, h, total = set(), [(0, 0)], 0   # (cost, point)
        while len(seen) < n:
            cost, i = heapq.heappop(h)
            if i in seen:
                continue
            seen.add(i)
            total += cost
            xi, yi = points[i]
            for j in range(n):
                if j not in seen:
                    xj, yj = points[j]
                    heapq.heappush(h, (abs(xi - xj) + abs(yi - yj), j))
        return total
```

743. Network Delay Time Medium

Problem: Signal sent from node k along weighted directed edges; time for all nodes to receive it (-1 if not all).
Key idea: Dijkstra from k; answer is the largest shortest-distance.
Complexity: O(E log V) time, O(V + E) space

```python
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        adj = defaultdict(list)
        for u, v, w in times:
            adj[u].append((v, w))
        dist, h = {}, [(0, k)]
        while h:
            d, u = heapq.heappop(h)
            if u in dist:
                continue
            dist[u] = d
            for v, w in adj[u]:
                if v not in dist:
                    heapq.heappush(h, (d + w, v))
        return max(dist.values()) if len(dist) == n else -1
```

778. Swim in Rising Water Hard

Problem: At time t you can swim through cells with height <= t. Earliest time to go from top-left to bottom-right.
Key idea: Dijkstra-like: min-heap keyed by the max height seen on the path so far.
Complexity: O(n^2 log n) time, O(n^2) space

```python
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        n = len(grid)
        h, seen = [(grid[0][0], 0, 0)], {(0, 0)}
        while h:
            t, r, c = heapq.heappop(h)
            if r == c == n - 1:
                return t
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < n and 0 <= nc < n and (nr, nc) not in seen:
                    seen.add((nr, nc))
                    heapq.heappush(h, (max(t, grid[nr][nc]), nr, nc))
```

269. Alien Dictionary Hard

Problem: Words are sorted in an unknown alphabet; return a valid letter order, or '' if inconsistent.
Key idea: Compare adjacent words: first differing letter gives an edge. Prefix trap: longer word before its prefix -> ''. Then topological sort.
Complexity: O(total chars) time, O(alphabet^2) space

```python
class Solution:
    def alienOrder(self, words: List[str]) -> str:
        adj = {c: set() for w in words for c in w}
        indeg = {c: 0 for c in adj}
        for w1, w2 in zip(words, words[1:]):
            if len(w1) > len(w2) and w1.startswith(w2):
                return
            for a, b in zip(w1, w2):
                if a != b:
                    if b not in adj[a]:
                        adj[a].add(b)
                        indeg[b] += 1
                    break
        q = deque(c for c in indeg if indeg[c] == 0)
        res = []
        while q:
            c = q.popleft()
            res.append(c)
            for nb in adj[c]:
                indeg[nb] -= 1
                if indeg[nb] == 0:
                    q.append(nb)
        return .join(res) if len(res) == len(adj) else
```

787. Cheapest Flights Within K Stops Medium

Problem: Cheapest price from src to dst using at most k stops (-1 if none).
Key idea: Bellman-Ford limited to k+1 rounds; update from a copy of last round's prices.
Complexity: O(k * E) time, O(n) space

```python
class Solution:
    def findCheapestPrice(self, n: int, flights: List[List[int]], src: int, dst: int, k: int) -> int:
        prices = [float(inf)] * n
        prices[src] = 0
        for _ in range(k + 1):
            tmp = prices[:]
            for u, v, w in flights:
                if prices[u] != float(inf) and prices[u] + w < tmp[v]:
                    tmp[v] = prices[u] + w
            prices = tmp
        return -1 if prices[dst] == float(inf) else prices[dst]
```

1-D Dynamic Programming · 12 problems

70. Climbing Stairs Easy

Problem: Ways to climb n steps taking 1 or 2 at a time.
Key idea: ways(n) = ways(n-1) + ways(n-2): Fibonacci with two variables.
Complexity: O(n) time, O(1) space

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

746. Min Cost Climbing Stairs Easy

Problem: Pay cost[i] to step from stair i (1 or 2 steps); start at 0 or 1; min cost to reach the top.
Key idea: dp[i] = cost[i] + min(dp[i+1], dp[i+2]); answer min(dp[0], dp[1]).
Complexity: O(n) time, O(1) space

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        a = b = 0                       # dp[i+1], dp[i+2]
        for i in range(len(cost) - 1, -1, -1):
            a, b = cost[i] + min(a, b), a
        return min(a, b)
```

198. House Robber Medium

Problem: Max money from houses in a line without robbing two adjacent ones.
Key idea: best(i) = max(best(i-1), best(i-2) + nums[i]).
Complexity: O(n) time, O(1) space

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        prev2 = prev1 = 0
        for n in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + n)
        return prev1
```

213. House Robber II Medium

Problem: Same as House Robber but houses form a circle (first and last are adjacent).
Key idea: Run House Robber on nums[1:] and nums[:-1], take the max (handle n == 1).
Complexity: O(n) time, O(1) space

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        def line(arr):
            a = b = 0
            for n in arr:
                a, b = b, max(b, a + n)
            return b
        if len(nums) == 1:
            return nums[0]
        return max(line(nums[1:]), line(nums[:-1]))
```

5. Longest Palindromic Substring Medium

Problem: Return the longest palindromic substring.
Key idea: Expand around every centre (odd: i,i and even: i,i+1).
Complexity: O(n^2) time, O(1) space

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        res =
        for i in range(len(s)):
            for l, r in ((i, i), (i, i + 1)):
                while l >= 0 and r < len(s) and s[l] == s[r]:
                    l, r = l - 1, r + 1
                if r - l - 1 > len(res):
                    res = s[l + 1 : r]
        return res
```

647. Palindromic Substrings Medium

Problem: Count all palindromic substrings.
Key idea: Expand around every centre; each successful expansion is one more palindrome.
Complexity: O(n^2) time, O(1) space

```python
class Solution:
    def countSubstrings(self, s: str) -> int:
        count = 0
        for i in range(len(s)):
            for l, r in ((i, i), (i, i + 1)):
                while l >= 0 and r < len(s) and s[l] == s[r]:
                    count += 1
                    l, r = l - 1, r + 1
        return count
```

91. Decode Ways Medium

Problem: Digits map 1->A ... 26->Z. How many ways can the digit string be decoded?
Key idea: dp[i] = dp[i+1] (if s[i] != '0') + dp[i+2] (if s[i:i+2] in 10..26).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def numDecodings(self, s: str) -> int:
        one, two = 1, 0                    # dp[i+1], dp[i+2]
        for i in range(len(s) - 1, -1, -1):
            cur = 0
            if s[i] != 0:
                cur = one
                if i + 1 < len(s) and 10 <= int(s[i : i + 2]) <= 26:
                    cur += two
            one, two = cur, one
        return one
```

322. Coin Change Medium

Problem: Fewest coins (unlimited supply) to make amount; -1 if impossible.
Key idea: dp[a] = 1 + min(dp[a - c]) over coins c (unbounded knapsack).
Complexity: O(amount * coins) time, O(amount) space

```python
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [0] + [amount + 1] * amount
        for a in range(1, amount + 1):
            for c in coins:
                if c <= a:
                    dp[a] = min(dp[a], 1 + dp[a - c])
        return dp[amount] if dp[amount] <= amount else -1
```

152. Maximum Product Subarray Medium

Problem: Largest product of a contiguous subarray.
Key idea: Track both current max and min product (a negative can flip min into max).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        res = max(nums)
        cur_max = cur_min = 1
        for n in nums:
            cand = (n, n * cur_max, n * cur_min)
            cur_max, cur_min = max(cand), min(cand)
            res = max(res, cur_max)
        return res
```

139. Word Break Medium

Problem: Can s be split into a sequence of dictionary words (reuse allowed)?
Key idea: dp[i] = True if some word w matches s[i:] start and dp[i + len(w)] is True.
Complexity: O(n * m * L) time, O(n) space

```python
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        dp = [False] * (len(s) + 1)
        dp[len(s)] = True
        for i in range(len(s) - 1, -1, -1):
            for w in wordDict:
                if s.startswith(w, i) and dp[i + len(w)]:
                    dp[i] = True
                    break
        return dp[0]
```

300. Longest Increasing Subsequence Medium

Problem: Length of the longest strictly increasing subsequence.
Key idea: Patience sorting: tails[k] = smallest tail of an increasing subsequence of length k+1; binary search to place each number.
Complexity: O(n log n) time, O(n) space

```python
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        tails = []
        for n in nums:
            i = bisect.bisect_left(tails, n)
            if i == len(tails):
                tails.append(n)
            else:
                tails[i] = n
        return len(tails)
```

416. Partition Equal Subset Sum Medium

Problem: Can the array be split into two subsets with equal sums?
Key idea: Need a subset summing to total/2: keep a set (or bitset) of reachable sums.
Complexity: O(n * sum) time, O(sum) space

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        total = sum(nums)
        if total % 2:
            return False
        target = total // 2
        reach = {0}
        for n in nums:
            reach |= {r + n for r in reach if r + n <= target}
            if target in reach:
                return True
        return target in reach
```

2-D Dynamic Programming · 11 problems

62. Unique Paths Medium

Problem: Paths from top-left to bottom-right of an m x n grid moving only right/down.
Key idea: paths(cell) = paths(below) + paths(right); keep one row.
Complexity: O(m * n) time, O(n) space

```python
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        row = [1] * n
        for _ in range(m - 1):
            for j in range(n - 2, -1, -1):
                row[j] += row[j + 1]
        return row[0]
```

1143. Longest Common Subsequence Medium

Problem: Length of the longest subsequence common to two strings.
Key idea: dp[i][j] = 1 + dp[i+1][j+1] if chars match else max(dp[i+1][j], dp[i][j+1]).
Complexity: O(m * n) time and space

```python
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                if text1[i] == text2[j]:
                    dp[i][j] = 1 + dp[i + 1][j + 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
        return dp[0][0]
```

309. Best Time to Buy and Sell Stock with Cooldown Medium

Problem: Unlimited trades, but after selling you must wait one day before buying. Max profit.
Key idea: State machine: hold, sold (cooldown today), rest. Transition each day.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        hold, sold, rest = float(-inf), 0, 0
        for p in prices:
            hold, sold, rest = max(hold, rest - p), hold + p, max(rest, sold)
        return max(sold, rest)
```

518. Coin Change II Medium

Problem: Number of combinations (order doesn't matter) of coins that make amount.
Key idea: Loop coins in the outer loop so each combination is counted once: dp[a] += dp[a - c].
Complexity: O(amount * coins) time, O(amount) space

```python
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        dp = [1] + [0] * amount
        for c in coins:
            for a in range(c, amount + 1):
                dp[a] += dp[a - c]
        return dp[amount]
```

494. Target Sum Medium

Problem: Put + or - before each number; count expressions that equal target.
Key idea: Map of reachable sum -> number of ways, updated number by number.
Complexity: O(n * sum) time, O(sum) space

```python
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        ways = {0: 1}
        for n in nums:
            nxt = defaultdict(int)
            for s, cnt in ways.items():
                nxt[s + n] += cnt
                nxt[s - n] += cnt
            ways = nxt
        return ways.get(target, 0)
```

97. Interleaving String Medium

Problem: Is s3 formed by interleaving s1 and s2 while keeping each one's order?
Key idea: dp[i][j] = can s1[i:] and s2[j:] form s3[i+j:].
Complexity: O(m * n) time and space

```python
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[m][n] = True
        for i in range(m, -1, -1):
            for j in range(n, -1, -1):
                if i < m and s1[i] == s3[i + j] and dp[i + 1][j]:
                    dp[i][j] = True
                if j < n and s2[j] == s3[i + j] and dp[i][j + 1]:
                    dp[i][j] = True
        return dp[0][0]
```

329. Longest Increasing Path in a Matrix Hard

Problem: Longest strictly increasing path moving in 4 directions.
Key idea: DFS + memo: lip(cell) = 1 + max lip(neighbour with larger value). No visited set needed (strictly increasing = no cycles).
Complexity: O(m * n) time and space

```python
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        R, C = len(matrix), len(matrix[0])
        @functools.lru_cache(None)
        def dfs(r, c):
            best = 1
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if 0 <= nr < R and 0 <= nc < C and matrix[nr][nc] > matrix[r][c]:
                    best = max(best, 1 + dfs(nr, nc))
            return best
        return max(dfs(r, c) for r in range(R) for c in range(C))
```

115. Distinct Subsequences Hard

Problem: How many distinct subsequences of s equal t?
Key idea: dp[i][j] = dp[i+1][j] (skip s[i]) + dp[i+1][j+1] (use it, if s[i] == t[j]).
Complexity: O(m * n) time and space

```python
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m, n = len(s), len(t)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][n] = 1
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                dp[i][j] = dp[i + 1][j]
                if s[i] == t[j]:
                    dp[i][j] += dp[i + 1][j + 1]
        return dp[0][0]
```

72. Edit Distance Hard

Problem: Minimum inserts, deletes and replacements to turn word1 into word2.
Key idea: If chars match, dp[i+1][j+1]; else 1 + min(insert dp[i][j+1], delete dp[i+1][j], replace dp[i+1][j+1]).
Complexity: O(m * n) time and space

```python
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1): dp[i][n] = m - i
        for j in range(n + 1): dp[m][j] = n - j
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                if word1[i] == word2[j]:
                    dp[i][j] = dp[i + 1][j + 1]
                else:
                    dp[i][j] = 1 + min(dp[i + 1][j], dp[i][j + 1], dp[i + 1][j + 1])
        return dp[0][0]
```

312. Burst Balloons Hard

Problem: Bursting balloon i earns left * i * right (current neighbours). Maximise total coins.
Key idea: Interval DP: choose which balloon in (l, r) is burst LAST; its neighbours are then fixed as l and r.
Complexity: O(n^3) time, O(n^2) space

```python
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        a = [1] + nums + [1]
        n = len(a)
        dp = [[0] * n for _ in range(n)]
        for length in range(2, n):
            for l in range(0, n - length):
                r = l + length
                for k in range(l + 1, r):
                    dp[l][r] = max(dp[l][r], a[l] * a[k] * a[r] + dp[l][k] + dp[k][r])
        return dp[0][n - 1]
```

10. Regular Expression Matching Hard

Problem: Full-string match supporting '.' (any char) and '*' (zero or more of the previous char).
Key idea: Top-down DP on (i, j). If p[j+1] == '*': skip the pair, or use it if first chars match.
Complexity: O(m * n) time and space

```python
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @functools.lru_cache(None)
        def dp(i, j):
            if j == len(p):
                return i == len(s)
            first = i < len(s) and p[j] in (s[i], .)
            if j + 1 < len(p) and p[j + 1] == *:
                return dp(i, j + 2) or (first and dp(i + 1, j))
            return first and dp(i + 1, j + 1)
        return dp(0, 0)
```

Greedy · 8 problems

53. Maximum Subarray Medium

Problem: Largest sum of a contiguous subarray.
Key idea: Kadane: drop the running sum when it goes negative.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        best, cur = nums[0], 0
        for n in nums:
            cur = max(cur, 0) + n
            best = max(best, cur)
        return best
```

55. Jump Game Medium

Problem: nums[i] is the max jump from i. Can you reach the last index?
Key idea: Walk backwards moving a 'goal' left whenever i + nums[i] >= goal.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        goal = len(nums) - 1
        for i in range(len(nums) - 2, -1, -1):
            if i + nums[i] >= goal:
                goal = i
        return goal == 0
```

45. Jump Game II Medium

Problem: Minimum jumps to reach the last index (always reachable).
Key idea: BFS by windows: [l, r] is reachable in 'jumps'; next window ends at the farthest reach.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def jump(self, nums: List[int]) -> int:
        jumps = l = r = 0
        while r < len(nums) - 1:
            farthest = max(i + nums[i] for i in range(l, r + 1))
            l, r = r + 1, farthest
            jumps += 1
        return jumps
```

134. Gas Station Medium

Problem: Circular route with gas[i] and cost[i]; find the unique start index to complete the loop, or -1.
Key idea: If total gas < total cost: -1. Otherwise reset the start to i+1 whenever the tank goes negative.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        if sum(gas) < sum(cost):
            return -1
        tank = start = 0
        for i in range(len(gas)):
            tank += gas[i] - cost[i]
            if tank < 0:
                tank, start = 0, i + 1
        return start
```

846. Hand of Straights Medium

Problem: Can the cards be split into groups of groupSize consecutive values?
Key idea: Always start a group at the smallest remaining card (min-heap / sorted keys) and consume the run.
Complexity: O(n log n) time, O(n) space

```python
class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        if len(hand) % groupSize:
            return False
        count = Counter(hand)
        for start in sorted(count):
            need = count[start]
            if need:
                for v in range(start, start + groupSize):
                    if count[v] < need:
                        return False
                    count[v] -= need
        return True
```

1899. Merge Triplets to Form Target Triplet Medium

Problem: Merging takes element-wise max. Can some merges produce exactly the target triplet?
Key idea: Ignore any triplet exceeding target anywhere; from the rest, check every target position is matched by someone.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int]) -> bool:
        good = set()
        for t in triplets:
            if t[0] > target[0] or t[1] > target[1] or t[2] > target[2]:
                continue
            for i in range(3):
                if t[i] == target[i]:
                    good.add(i)
        return len(good) == 3
```

763. Partition Labels Medium

Problem: Split s into as many parts as possible so each letter appears in only one part; return part sizes.
Key idea: Record each letter's last index; extend the current part's end to the max last index seen; cut when i == end.
Complexity: O(n) time, O(26) space

```python
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        last = {c: i for i, c in enumerate(s)}
        res, size, end = [], 0, 0
        for i, c in enumerate(s):
            size += 1
            end = max(end, last[c])
            if i == end:
                res.append(size)
                size = 0
        return res
```

678. Valid Parenthesis String Medium

Problem: '(' , ')' and '*' (which may be '(', ')' or empty). Can the string be valid?
Key idea: Track a range [lo, hi] of possible open counts; hi $<$ 0 -$>$ invalid; clamp lo at 0; valid if lo == 0 at end.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def checkValidString(self, s: str) -> bool:
        lo = hi = 0
        for c in s:
            if c == (:
                lo, hi = lo + 1, hi + 1
            elif c == ):
                lo, hi = lo - 1, hi - 1
            else:
                lo, hi = lo - 1, hi + 1
            if hi < 0:
                return False
            lo = max(lo, 0)
        return lo == 0
```

Intervals · 6 problems

57. Insert Interval Medium

Problem: Insert a new interval into a sorted, non-overlapping list and merge as needed.
Key idea: Three phases: copy intervals ending before it; merge all overlapping into it; copy the rest.
Complexity: O(n) time, O(n) space

```
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        res, i, n = [], 0, len(intervals)
        while i < n and intervals[i][1] < newInterval[0]:
            res.append(intervals[i]); i += 1
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval = [min(newInterval[0], intervals[i][0]),
                           max(newInterval[1], intervals[i][1])]
            i += 1
        res.append(newInterval)
        res.extend(intervals[i:])
        return res
```

56. Merge Intervals Medium

Problem: Merge all overlapping intervals.
Key idea: Sort by start; extend the last merged interval if the next one starts before it ends.
Complexity: O(n log n) time, O(n) space

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort(key=lambda x: x[0])
        res = [intervals[0][:]]
        for s, e in intervals[1:]:
            if s <= res[-1][1]:
                res[-1][1] = max(res[-1][1], e)
            else:
                res.append([s, e])
        return res
```

435. Non-overlapping Intervals Medium

Problem: Minimum intervals to remove so the rest don't overlap.
Key idea: Sort by start; on overlap, remove the one with the larger end (keep the earlier-ending one).
Complexity: O(n log n) time, O(1) space

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort()
        removed, prev_end = 0, intervals[0][1]
        for s, e in intervals[1:]:
            if s >= prev_end:
                prev_end = e
            else:
                removed += 1
                prev_end = min(prev_end, e)
        return removed
```

252. Meeting Rooms Easy

Problem: Can one person attend all meetings (no overlaps)?
Key idea: Sort by start; any meeting starting before the previous ends is a conflict.
Complexity: O(n log n) time, O(1) space

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        for i in range(1, len(intervals)):
            if intervals[i][0] < intervals[i - 1][1]:
                return False
        return True
```

253. Meeting Rooms II Medium

Problem: Minimum number of rooms needed for all meetings.
Key idea: Sort starts and ends separately; sweep: a start before the earliest end needs a new room.
Complexity: O(n log n) time, O(n) space

```python
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        starts = sorted(i[0] for i in intervals)
        ends = sorted(i[1] for i in intervals)
        rooms = best = e = 0
        for s in starts:
            while e < len(ends) and ends[e] <= s:
                rooms -= 1
                e += 1
            rooms += 1
            best = max(best, rooms)
        return best
```

1851. Minimum Interval to Include Each Query Hard

Problem: For each query point, return the size of the smallest interval containing it (-1 if none).
Key idea: Process queries sorted; push intervals whose start <= q into a min-heap by size; pop those ending before q.
Complexity: O((n + q) log n) time, O(n + q) space

```python
class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        intervals.sort()
        h, ans, i = [], {}, 0
        for q in sorted(queries):
            while i < len(intervals) and intervals[i][0] <= q:
                l, r = intervals[i]
                heapq.heappush(h, (r - l + 1, r))
                i += 1
            while h and h[0][1] < q:
                heapq.heappop(h)
            ans[q] = h[0][0] if h else -1
        return [ans[q] for q in queries]
```

Math & Geometry · 8 problems

48. Rotate Image Medium

Problem: Rotate an n x n matrix 90 degrees clockwise in place.
Key idea: Transpose, then reverse each row.
Complexity: O(n^2) time, O(1) space

```python
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        for row in matrix:
            row.reverse()
```

54. Spiral Matrix Medium

Problem: Return all elements in clockwise spiral order.
Key idea: Shrink four boundaries (top, bottom, left, right) after each side is read.
Complexity: O(m * n) time, O(1) extra space

```python
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        res = []
        top, bottom, left, right = 0, len(matrix) - 1, 0, len(matrix[0]) - 1
        while top <= bottom and left <= right:
            for c in range(left, right + 1): res.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1): res.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1): res.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): res.append(matrix[r][left])
                left += 1
        return res
```

73. Set Matrix Zeroes Medium

Problem: If a cell is 0, set its entire row and column to 0, in place.
Key idea: Use the first row/column as marker arrays; keep a separate flag for row 0.
Complexity: O(m * n) time, O(1) space

```python
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        R, C = len(matrix), len(matrix[0])
        row0_zero = any(matrix[0][c] == 0 for c in range(C))
        for r in range(1, R):
            for c in range(C):
                if matrix[r][c] == 0:
                    matrix[0][c] = 0
                    matrix[r][0] = 0
        for r in range(1, R):
            for c in range(1, C):
                if matrix[0][c] == 0 or matrix[r][0] == 0:
                    matrix[r][c] = 0
        if matrix[0][0] == 0:
            for r in range(R):
                matrix[r][0] = 0
        if row0_zero:
            for c in range(C):
                matrix[0][c] = 0
```

202. Happy Number Easy

Problem: Repeatedly replace n with the sum of squares of its digits. Does it reach 1?
Key idea: Either reaches 1 or loops: detect the loop with a set (or Floyd).
Complexity: O(log n) per step, O(log n) space

```python
class Solution:
    def isHappy(self, n: int) -> bool:
        seen = set()
        while n != 1 and n not in seen:
            seen.add(n)
            n = sum(int(d) ** 2 for d in str(n))
        return n == 1
```

66. Plus One Easy

Problem: Add one to a number stored as a list of digits.
Key idea: From the end: 9 becomes 0 and carry; otherwise increment and return. All 9s -> prepend 1.
Complexity: O(n) time, O(1) space

```python
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        for i in range(len(digits) - 1, -1, -1):
            if digits[i] < 9:
                digits[i] += 1
                return digits
            digits[i] = 0
        return [1] + digits
```

50. Pow(x, n) Medium

Problem: Compute x raised to integer n (n may be negative).
Key idea: Fast exponentiation: square x and halve n; multiply in x when the low bit is 1.
Complexity: O(log n) time, O(1) space

```python
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x, n = 1 / x, -n
        res = 1.0
        while n:
            if n & 1:
                res *= x
            x *= x
            n >>= 1
        return res
```

43. Multiply Strings Medium

Problem: Multiply two non-negative numbers given as strings, without converting to int.
Key idea: Grade-school multiplication: digit i times digit j lands at position i + j (on reversed strings).
Complexity: O(m * n) time, O(m + n) space

```python
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == 0 or num2 == 0:
            return 0
        res = [0] * (len(num1) + len(num2))
        a, b = num1[::-1], num2[::-1]
        for i in range(len(a)):
            for j in range(len(b)):
                res[i + j] += int(a[i]) * int(b[j])
                res[i + j + 1] += res[i + j] // 10
                res[i + j] %= 10
        while len(res) > 1 and res[-1] == 0:
            res.pop()
        return .join(map(str, res[::-1]))
```

2013. Detect Squares Medium

Problem: add(point) to a multiset; count(point) returns how many axis-aligned squares it completes with stored points.
Key idea: Treat each stored point as the diagonal corner; if it forms a square, multiply counts of the other two corners.
Complexity: add O(1), count O(n) time; O(n) space

```python
class DetectSquares:
    def __init__(self):
        self.pts = Counter()

    def add(self, point: List[int]) -> None:
        self.pts[tuple(point)] += 1

    def count(self, point: List[int]) -> int:
        px, py = point
        res = 0
        for (x, y), c in list(self.pts.items()):
            if abs(px - x) != abs(py - y) or x == px or y == py:
                continue
            res += c * self.pts[(x, py)] * self.pts[(px, y)]
        return res
```

Bit Manipulation · 7 problems

136. Single Number Easy

Problem: Every value appears twice except one; find it in O(1) space.
Key idea: XOR everything: pairs cancel (a ^ a = 0).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        res = 0
        for n in nums:
            res ^= n
        return res
```

191. Number of 1 Bits Easy

Problem: Count the set bits of an unsigned integer.
Key idea: n &= n - 1 clears the lowest set bit; count iterations.
Complexity: O(number of set bits) time, O(1) space

```python
class Solution:
    def hammingWeight(self, n: int) -> int:
        count = 0
        while n:
            n &= n - 1
            count += 1
        return count
```

338. Counting Bits Easy

Problem: For every i in 0..n, return the number of 1 bits in i.
Key idea: dp[i] = dp[i >> 1] + (i & 1).
Complexity: O(n) time, O(n) space

```python
class Solution:
    def countBits(self, n: int) -> List[int]:
        dp = [0] * (n + 1)
        for i in range(1, n + 1):
            dp[i] = dp[i >> 1] + (i & 1)
        return dp
```

190. Reverse Bits Easy

Problem: Reverse the bits of a 32-bit unsigned integer.
Key idea: Take bit i from n and place it at position 31 - i.
Complexity: O(32) time, O(1) space

```python
class Solution:
    def reverseBits(self, n: int) -> int:
        res = 0
        for i in range(32):
            bit = (n >> i) & 1
            res |= bit << (31 - i)
        return res
```

268. Missing Number Easy

Problem: Array holds n distinct numbers from 0..n; find the missing one.
Key idea: XOR all indices 0..n with all values (or n(n+1)/2 - sum).
Complexity: O(n) time, O(1) space

```python
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        res = len(nums)
        for i, n in enumerate(nums):
            res ^= i ^ n
        return res
```

371. Sum of Two Integers Medium

Problem: Add two integers without using + or -.
Key idea: sum = a ^ b, carry = (a & b) << 1; repeat. In Python, mask to 32 bits and fix the sign at the end.
Complexity: O(32) time, O(1) space

```python
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        while b & mask:
            a, b = a ^ b, (a & b) << 1
        a &= mask
        return a if a <= 0x7FFFFFFF else ~(a ^ mask)
```

7. Reverse Integer Medium

Problem: Reverse the digits of a 32-bit signed integer; return 0 if the result overflows.
Key idea: Pop digits with % and //, building the result; check the 32-bit bounds.
Complexity: O(log x) time, O(1) space

```python
class Solution:
    def reverse(self, x: int) -> int:
        sign = -1 if x < 0 else 1
        x, res = abs(x), 0
        while x:
            res = res * 10 + x % 10
            x //= 10
        res *= sign
        return res if -2**31 <= res <= 2**31 - 1 else 0
```